



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

RELATÓRIO

COMPILADOR DIDÁTICO EM C

(Parser, Pilha, AST — Módulo do Elemento 2)

TURMA	EINF4 T2
GRUPO	Nº2
Nº DE MATRÍCULA	NOME DO ALUNO
20241039	Njila Camissombo
20231023	António Monteiro
20242833	Ventura Manuel

Funções dos membros do grupo

Nome	Função
Njila Camissombo	- Implementação do Parser (parser.c / parser.h); - Implementação da Pilha (stack.c / stack.h); - Implementação da AST (ast.c / ast.h);
Ventura Manuel	- Análise Semântica (semantic.c / semantic.h); - Tabela de Símbolos (hashtable, BST, AVL); - Gerador de Código (generator.c / generator.h);
António Monteiro	- Implementação do menu principal (main.c); - Integração dos módulos do compilador; - Coordenação do fluxo geral do programa.

Índice

. Introdução	4
. Metodologia	5
1. Gramática Definida	6
2. Módulo Parser (parser.c / parser.h)	7
3. Módulo Pilha (stack.c / stack.h)	8
4. Módulo AST (ast.c / ast.h)	9
5. Módulo Semântica + Tabelas (Elemento 3)	10
6. Bibliotecas Utilizadas	11
. Conclusão	12
. Referências	13

Introdução

O presente relatório descreve o desenvolvimento de um compilador didático em linguagem C, elaborado no âmbito da disciplina de Estrutura de Dados II (EDI). O projeto foi realizado em grupo, com o objetivo de aplicar, de forma prática e integrada, os conhecimentos teóricos adquiridos ao longo das aulas, nomeadamente no domínio das estruturas de dados, algoritmos e teoria de compiladores.

Um compilador é um programa que traduz código escrito numa linguagem de alto nível (linguagem fonte) para uma representação que pode ser executada por um computador. Segundo Cormen et al. (2022), a eficiência de um compilador depende diretamente das estruturas de dados subjacentes utilizadas em cada fase de análise. Aho, Hopcroft e Ullman (1983) estabelecem que as fases fundamentais de um compilador são: análise léxica, análise sintática, análise semântica e geração de código — exactamente as fases implementadas neste projecto.

O compilador desenvolvido suporta uma linguagem simplificada com declaração de variáveis inteiras, atribuições, operações aritméticas, controlo de fluxo (if/while) e a instrução de saída print. Cada elemento do grupo foi responsável por um módulo distinto, garantindo a integração coesa entre as partes através de estruturas e tipos partilhados.

Este relatório documenta de forma detalhada a arquitetura do sistema, as decisões de design tomadas, os algoritmos implementados, os testes realizados e os resultados obtidos, servindo simultaneamente como documentação técnica do projecto.

Metodologia

O desenvolvimento do compilador seguiu uma abordagem modular e incremental. A equipa começou por definir a gramática formal da linguagem alvo, que serviu de contrato entre todos os módulos. De seguida, cada elemento implementou o seu módulo de forma independente, com interfaces bem definidas, e a integração foi feita numa fase posterior.

Abordagem de desenvolvimento

- Fase 1 — Definição da gramática e tipos partilhados (tipos.h): todos os membros participaram nesta fase para garantir compatibilidade entre módulos.
- Fase 2 — Implementação independente de cada módulo, com testes unitários individuais.
- Fase 3 — Integração progressiva: Lexer → Parser → Semântica → Gerador.
- Fase 4 — Testes de sistema com programas fonte completos.

Técnica de análise sintática adoptada

A técnica escolhida foi o Parser Descendente Recursivo (Recursive Descent Parser). Segundo Bullinaria (2019), esta é uma das abordagens mais intuitivas para implementar um parser LL(1), onde cada regra da gramática é mapeada directamente para uma função em C. A principal vantagem é a clareza do código e a facilidade de depuração, uma vez que a estrutura do parser espelha a estrutura da gramática.

A escolha desta técnica é suportada por Aho, Hopcroft e Ullman (1983), que descrevem o parser descendente recursivo como adequado para gramáticas sem ambiguidade e sem recursão à esquerda — características que foram garantidas na definição da gramática deste compilador.

1. Gramática Definida

Antes de implementar o parser, foi necessário formalizar a gramática da linguagem suportada pelo compilador. A gramática define quais sequências de tokens são consideradas válidas.

Regras da Gramática (BNF simplificado)

```
programa    → "main" "(" ")" bloco
bloco       → "{" lista_instrucoes "}"
lista_instrucoes → instrucao lista_instrucoes | ε
instrucao   → declaracao_var | atribuicao | print_stmt | if_stmt | while_stmt
declaracao_var → "int" IDENTIFICADOR ";"
atribuicao   → IDENTIFICADOR "=" expressao ";"
print_stmt  → "print" "(" expressao ")" ";"
if_stmt     → "if" "(" condicao ")" bloco
while_stmt  → "while" "(" condicao ")" bloco
condicao     → expressao op_rel expressao
op_rel      → "==" | "!=" | "<" | ">"
expressao   → termo ( "+" | "-" ) termo *
termo       → IDENTIFICADOR | NUMERO
```

Exemplos de entrada válida e inválida

Código	Resultado	Motivo
int x;	✓ Aceite	Segue: int IDENT ;
int ;	✗ Erro	Falta o identificador
x = a + 5;	✓ Aceite	Atribuição válida
x = + 5;	✗ Erro	Expressão inválida
print(x);	✓ Aceite	Parênteses corretos
print(x}	✗ Erro	Fechamento incorreto: } em vez de)

2. Módulo Parser (parser.c / parser.h)

O parser é o núcleo deste módulo. Recebe a lista de tokens produzida pelo Lexer (Elemento 1) e verifica se a sequência respeita a gramática definida. Não verifica tipos — apenas a estrutura sintática.

Ficheiros

- parser.h — declara os tipos Token, TipoToken e as funções públicas parser_init() e parse_programa()
- parser.c — implementa o parser descendente recursivo, com uma função por regra da gramática

Funções implementadas

Função	Descrição
parser_init(tokens, total)	Inicializa o parser com a lista de tokens
parse_programa()	Ponto de entrada — regra raiz da gramática
parse_bloco()	Analisa um bloco { ... }
parse_instrucao()	Despacha para o tipo correto de instrução
parse_declaracao_var()	Analisa: int IDENTIFICADOR ;
parse_atribuicao()	Analisa: IDENT = expressao ;
parse_print_stmt()	Analisa: print (expressao) ;
parse_if_stmt()	Analisa: if (condicao) bloco
parse_while_stmt()	Analisa: while (condicao) bloco
parse_expressao()	Analisa somas e subtrações
parse_termo()	Analisa identificadores e números
consume(tipo_esperado)	Consome o token atual, ativa a pilha se necessário

Em caso de erro sintático, o parser imprime uma mensagem clara indicando a linha, o token esperado e o token encontrado, e termina a execução.

3. Módulo Pilha (stack.c / stack.h)

A pilha de caracteres é utilizada pelo parser para verificar se os delimitadores de abertura '(' e '{' são corretamente fechados por ')' e '}', respetivamente.

Lógica de funcionamento

- Quando o parser consome '(' ou '{', empurra o caractere na pilha (push)
- Quando consome ')' ou '}', retira da pilha (pop) e verifica se o par é correto
- No final do programa, verifica se a pilha está vazia — se não estiver, há um delimitador por fechar

Exemplo

```
print(x); → push('(') → pop('(') → par correto ✓  
print(x} → push('(') → pop('{') → par errado ✗
```

Funções implementadas

Função	Descrição
<code>pilha_init(p)</code>	Inicializa a pilha (topo = -1)
<code>pilha_push(p, c)</code>	Empurra o caractere c para o topo
<code>pilha_pop(p)</code>	Remove e devolve o caractere do topo
<code>pilha_topo(p)</code>	Espia o topo sem remover
<code>pilha_vazia(p)</code>	Devolve 1 se a pilha estiver vazia
<code>pilha_verificar_fim(p)</code>	Verifica se todos os delimitadores foram fechados

4. Módulo AST (ast.c / ast.h)

A Árvore Sintática Abstrata (AST) é o resultado final produzido pelo parser. Representa a estrutura hierárquica do programa de forma que o módulo de análise semântica (Elemento 3) possa percorrer e analisar.

Exemplo de transformação

O código fonte:

```
x = a + 5;
```

É transformado na seguinte árvore:

```
    ATRIB(=)
   /   \
IDENT(x) OPER(+)
      /   \
    IDENT(a) NUM(5)
```

Tipos de nós (TipoNo)

Tipo	Representa
NO_PROGRAMA	Raiz da árvore — o programa inteiro
NO_BLOCO	Bloco { ... }
NO_DECLARACAO_VAR	Declaração de variável: int x;
NO_ATRIBUICAO	Atribuição: x = expr;
NO_OPERACAO	Operação aritmética: +, -, *, /
NO_IDENTIFICADOR	Nome de variável
NO_NUMERO	Literal numérico
NO_PRINT	Instrução print(expr);
NO_IF	Instrução if (cond) bloco
NO_WHILE	Instrução while (cond) bloco
NO_CONDICAO	Condição relacional: ==, !=, <, >

Funções implementadas

- ast_criar_no(tipo, valor) — aloca e inicializa um novo nó
- ast_adicionar_filho(pai, filho) — liga o filho ao pai
- ast_imprimir(no, profundidade) — imprime a árvore de forma indentada
- ast_libertar(no) — liberta recursivamente toda a memória

- `ast_tipo_nome(tipo)` — devolve o nome legível do tipo de nó (para debug)

5. Módulo Semântica + Tabelas

O Elemento 3 recebe a AST produzida pelo parser e realiza a análise semântica do programa. Para além da verificação de significado, é responsável pela geração do código intermédio.

5.1 Tabela de Símbolos

Os símbolos do programa (variáveis declaradas) são armazenados em três estruturas complementares:

- Hash Table (hashtable.c / hashtable.h) — permite inserção, pesquisa e remoção em tempo $O(1)$ médio
- BST — Binary Search Tree (bst.c / bst.h) — organiza os símbolos por ordem alfabética
- AVL (avl.c / avl.h) — BST auto-balanceada que garante pesquisa eficiente $O(\log n)$

5.2 Análise Semântica (semantic.c / semantic.h)

A análise semântica percorre a AST e verifica:

- Variável não declarada — erro se `x = 10` aparece sem `int x` antes
- Variável duplicada — erro se `int x` aparece duas vezes
- Tipos incompatíveis — erro se `int x` recebe um valor de tipo `string`

5.3 Gerador de Código (generator.c / generator.h)

O gerador percorre a AST e produz instruções de código intermédio. Exemplo:

```
x = a + 5;  →  LOAD a
              ADD 5
              STORE x
```

5.4 Relatório Final

No fim da compilação, o sistema apresenta um relatório resumido:

```
Tokens processados: 50
Erros léxicos:      0
Erros sintáticos:   1
Erros semânticos:   0
Símbolos declarados: x (int), a (int)
```

6. Bibliotecas Utilizadas no Programa

`#include <stdio.h>`

Utilizada para exibir mensagens no terminal e para capturar a entrada de dados do utilizador. Funções como `printf()` e `fprintf()` são essenciais para reportar erros sintáticos e imprimir a AST.

`#include <stdlib.h>`

Utilizada para alocação dinâmica de memória através de `malloc()` e `free()`. É fundamental para a criação dos nós da AST e dos elementos da pilha, uma vez que o número de nós não é conhecido em tempo de compilação.

`#include <string.h>`

Utilizada para manipulação de strings, permitindo operações como cópia (`strcpy`, `strncpy`), comparação (`strcmp`) e concatenação. Necessária para copiar os valores dos tokens para os nós da AST.

Conclusão

O desenvolvimento do compilador didático em C permitiu aplicar de forma prática conceitos fundamentais de estrutura de dados e teoria de compiladores, nomeadamente análise sintática, pilhas, árvores e tabelas de dispersão.

O módulo desenvolvido pelo Elemento 2 — Parser, Pilha e AST — cumpre o objetivo de receber os tokens do Lexer, verificar a estrutura sintática do programa e produzir uma AST coerente para uso pelo módulo de análise semântica.

A divisão modular do projeto revelou-se uma estratégia eficaz: cada elemento trabalhou de forma independente, com interfaces bem definidas, facilitando a integração final. A utilização do parser descendente recursivo, aliada à pilha de delimitadores e à construção incremental da AST, resultou num módulo robusto, bem estruturado e fácil de depurar.

Este projeto contribuiu significativamente para a compreensão prática de como um compilador real funciona, evidenciando a importância das estruturas de dados na resolução de problemas complexos do mundo real.

Referências

Livros e publicações acadêmicas:

- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4ª edição. Cambridge, Massachusetts: The MIT Press, 2022. ISBN 978-0-262-04630-5.
- AHO, Alfred V.; HOPCROFT, John E.; ULLMAN, Jeffrey D. Data Structures and Algorithms. Reading, Massachusetts: Addison-Wesley, 1983.
- BULLINARIA, John A. Lecture Notes for Data Structures and Algorithms. Revised edition. Birmingham, UK: School of Computer Science, University of Birmingham, 2019.

Recursos online:

- CODECRUSH. Estrutura de Dados. Disponível em: <https://codecrush.com.br/blog/estrutura-de-dados>
- STUDOCU. Tema 4: Listas, Pilhas, Filas e Deques. Disponível em: <https://www.studocu.com/pt-br/document/centro-universitario-estacio-de-sao-paulo/ciencia-de-dados/tema-4-listas-pilhas-filas-e-deques/86599879>